

# Databases 51

## SPARQL Query optimization

- Comparing costs of Plans
- Cost of filters
- Join Order Optimization

# Query evaluation: single triple patterns

- Evaluation of single triple pattern is a primitive offered by a triplestore :
- `SELECT * WHERE {C45678 ?predicate ?object }`
- Possible 8 *abstract triple patterns*:

|                       |                         |                      |                  |
|-----------------------|-------------------------|----------------------|------------------|
| <code>?subject</code> | <code>?predicate</code> | <code>?object</code> | <code>XYZ</code> |
| <code>C45678</code>   | <code>?predicate</code> | <code>?object</code> | <code>SYZ</code> |
| <code>?subject</code> | <code>?predicate</code> | <code>SOFTENG</code> | <code>XYO</code> |
| <code>?subject</code> | <code>programme</code>  | <code>?object</code> | <code>XPZ</code> |
| <code>?subject</code> | <code>programme</code>  | <code>SOFTENG</code> | <code>XPO</code> |
| <code>C45678</code>   | <code>programme</code>  | <code>?object</code> | <code>SPZ</code> |
| <code>C45678</code>   | <code>?predicate</code> | <code>SOFTENG</code> | <code>SYO</code> |
| <code>C45678</code>   | <code>programme</code>  | <code>SOFTENG</code> | <code>SPO</code> |



Most specific

- Optimal query cost for single triple pattern: linear in size of result set
- Is achievable with enough index support! (but not always done)

# Cost with larger basic graph pattern.

- avg 2.2 (?lecturer teaches ?course) avg 4.2
- 1 (?lecturer assesses ?course) avg 1.9
- 40 (?course programme SOFTENG)
- 60 (?course programme GENED)
- **SELECT \* {  
  ?course2 programme GENED  
  ?lecturer teaches ?course2 .  
  ?lecturer assesses ?course1 .  
  ?course1 programme SOFTENG .  
}**

# Cost with larger basic graph pattern.

- avg 2.2 (?lecturer teaches ?course) avg 4.2
- 1 (?lecturer assesses ?course) avg 1.9
- 40 (?course programme SOFTENG)
- 60 (?course programme GENED)

```
SELECT * {  
  ?course2 programme GENED          60          = 60  
  ?lecturer teaches ?course2 .      *2.2        = 132  
  ?lecturer assesses ?course1 .     *1.9        < 255  
  ?course1 programme SOFTENG .      *1          < 255  
}
```

**Abstract Triple Pattern SPO** **Conservative est.** < 702

## Cost after changing order

- avg 2.2 (?lecturer teaches ?course) avg 4.2
- 1 (?lecturer assesses ?course) avg 1.9
- 40 (?course programme SOFTENG)
- 60 (?course programme GENED)

[illegible]

# Abstract Triple Pattern SPO

# To rule an option out we need lower bound

- avg 2.2 (?lecturer teaches ?course) avg 4.2
- 1 (?lecturer assesses ?course) avg 1.9
- 40 (?course programme SOFTENG)
- 60 (?course programme GENED)

```
• SELECT * {  
  ?course2 programme GENED          60          = 60  
  ?lecturer teaches ?course2 .      *2.2        = 132  
  ?lecturer assesses ?course1 .      *1.9        > 240  
  ?course1 programme SOFTENG .       *1 > 240  
}
```

---

> 650

Lower bound

→

# Cost estimation with filters

- avg 2.2 (?lecturer teaches ?course) avg 4.2
- 1 (?lecturer assesses ?course) avg 1.9
- 40 (?course programme SOFTENG)
- 60 (?course programme GENED)
- Courses with courseNumber 700 or higher: 20 percent
- PREFIX uni: <http://www.auckland.ac.nz/all/>

**Cost  
metric for  
filter**

```
SELECT * {  
  ?course1 uni:programme uni:SOFTENG .      40      = 40  
  ?lecturer uni:assesses ?course1 .          *1      = 40  
  ?lecturer uni:teaches ?course2 .           *4.2    = 168  
  ?course2 uni:programme uni:GENED            *1      = 168  
  ?course1 uni:cnr ?coursenr . Can we do *1      = 168  
  FILTER (?coursenr >= 700) better? *1      = 168  
}
```

752

# Cost estimation with filters

- avg 2.2 (?lecturer teaches ?course) avg 4.2
- 1 (?lecturer assesses ?course) avg 1.9
- 40 (?course programme SOFTENG)
- 60 (?course programme GENED)
- Courses with courseNumber 700 or higher: 20 percent

```

SELECT * {
  ?course1 uni:programme SOFTENG .           40      = 40
  ?lecturer uni:assesses ?course1 .          *1      = 40
  ?course1 uni:cnr ?coursenr .                 *1      = 40
  FILTER (?coursenr >= 700)                    *1      = 40
  ?lecturer uni:teaches ?course2 .  *0.2  *4.2    ~ 34
  ?course2 uni:programme uni:GENED             *1      ~ 34
}                                              ~ 228

```



# Cost estimation with filters

- avg 2.2 (?lecturer teaches ?course) avg 4.2
- 1 (?lecturer assesses ?course) avg 1.9
- 40 (?course programme SOFTENG)
- 60 (?course programme GENED)
- Courses with courseNumber 700 or higher: 20 percent

```

SELECT * {
  ?course1 uni:programme SOFTENG .           40      = 40
  ?course1 uni:cnr ?coursenr .                *1      = 40
  FILTER (?coursenr >= 700)                    *1      = 40
  ?lecturer uni:assesses ?course1 . *0.2 *1    = 8
  ?lecturer uni:teaches ?course2 .            *4.2    < 38
  ?course2 uni:programme uni:GENED              *1    < 38
}                                              < 204

```



# Lower bound

- avg 2.2 (?lecturer teaches ?course) avg 4.2
- 1 (?lecturer assesses ?course) avg 1.9
- 40 (?course programme SOFTENG)
- 60 (?course programme GENED)
- Courses with courseNumber 700 or higher: 20 percent

```

SELECT * {
  ?course1 uni:programme SOFTENG .           40      = 40
  ?lecturer uni:assesses ?course1 .         *1      = 40
  ?course1 uni:cnr ?coursenr .               *1      = 40
  FILTER (?coursenr >= 700)                  *1      = 40
  ?lecturer uni:teaches ?course2 . *0.2 *4.2  > 32
  ?course2 uni:programme uni:GENED           *1      > 32
}                                              > 224

```



# Summary

- **Query optimization: Join order optimization**
- The order of the triple patterns in SPARQL give a simple linear query execution plan.
- Depending on the order of the triples in this plan the cost can differ.
- Previous choices in the plan bind variables and change the effective abstract triple pattern.
- With cost metrics we can estimate cost of query.
- To exclude a plan we need a lower bound, do accept a plan we want a conservative estimate.
- Applying filters early is good – reducing the cost.

("pat.lee" order ?shop) 20  
 avg 10000 (?customer order ?shop) avg 10  
 avg 100 (?shop stocks ?product) avg 1000  
 avg 100 (?product contains ?ingredient) avg 10  
 avg 1000 (?product class class ) 1  
 avg 1000 (?product class ORGANIC ) avg 0.1  
 avg 10 (?ingredient description ?name ) avg 1  
 10 (?ingredient description "Waiheke Honey" ) avg 0.01

```

SELECT ?product {
  "pat.lee" order ?shop.                = 20
  ?shop stocks ?product.                * 1000    = 20 000
  ?product contains ?ingredient.        * 10      = 200 000
  ?product class ORGANIC.                * 1       = 200 000
  ?ingredient description "Waiheke Honey". * 1     = 200 000
                                           = 620 020

```

("pat.lee" order ?shop) 20

avg 10000 (?customer order ?shop) avg 10

avg 100 (?shop stocks ?product) avg 1000

avg 100 (?product contains ?ingredient) avg 10

avg 1000 (?product class class ) 1

avg 1000 (?product class ORGANIC ) avg 0.1

avg 10 (?ingredient description ?name ) avg 1

10 (?ingredient description "Waiheke Honey" ) avg 0.01

organic

**SELECT** ?product {

"pat.lee" order ?shop. 20

?shop stocks ?product.

1000.

?product contains ?ingredient.

10.

?product class ORGANIC.

1

?ingredient description "Waiheke Honey". 1 }

("pat.lee" order ?shop) 20

avg 10000 (?customer order ?shop) avg 10

avg 100 (?shop stocks ?product) avg 1000

avg 100 (?product contains ?ingredient) avg 10

avg 1000 (?product class class ) 1

avg 1000 (?product class ORGANIC ) avg 0.1

avg 10 (?ingredient description ?name ) avg 1

10 (?ingredient description "Waiheke Honey" ) avg 0.01

**SELECT ?product {**

"pat.lee" order ?shop. = 20

?shop stocks ?product. \* 1000 = 20 000

?product class ORGANIC. \* 1 = 20 000

?product contains ?ingredient. \* 0.1 \* 10 = 20 000

?ingredient description "Waiheke Honey". = 20 000

= 80 020

("pat.lee" order ?shop) 20

avg 10000 (?customer order ?shop) avg 10

avg 100 (?shop stocks ?product) avg 1000

avg 100 (?product contains ?ingredient) avg 10

avg 1000 (?product class class ) 1

avg 1000 (?product class ORGANIC ) avg 0.1

avg 10 (?ingredient description ?name ) avg 1

10 (?ingredient description "Waiheke Honey" ) avg 0.01

**SELECT ?product {**

?ingredient description "Waiheke Honey". = 10

?product contains ?ingredient. \* 100 = 1000

?product class ORGANIC. \* 1 = 1000

?shop stocks ?product. \* 0.1 \* 100 = 10 000

"pat.lee" order ?shop. = 10 000

**Best Plan found**

= 22010

